Graph-Based Intrusion Detection: Detecting Network Attacks with Graph Neural Networks and LLM-Generated Alert Explanations

Shiyi Liu (Team Leader)
Dept. of Computer Science
Virginia Tech
lshiyi25@vt.edu

Vighnesh Nair
Dept. of Computer Science
Virginia Tech
vighneshnair@vt.edu

Krishna Inukonda
Dept. of Computer Science
Virginia Tech
kinukond@vt.edu

Gokulakrishnan Ravi
Dept. of Computer Science
Virginia Tech
gravi@vt.edu

***Abstract*—Traditional network intrusion detection systems (NIDS) classify network flows independently, often overlooking communication patterns among hosts. This project develops a graph-based intrusion detection framework using NF-UNSW-NB15-v2 and NF-CICIDS2018-v3. Network flows are represented as attributed edges between IP-address nodes in fixed-size windows of 5,000 flows. We implement an edge-aware graph neural network based on E-GraphSAGE and compare it against Random Forest, XGBoost, and MLP baselines. We also investigate attention-based aggregation and cross-dataset generalization. An LLM explanation layer will generate human-readable alerts from detected malicious flows and their graph context. Evaluation emphasizes macro and per-class F1 scores to account for class imbalance.**

***Index Terms*—intrusion detection, graph neural networks, network security, deep learning, edge classification, domain generalization, large language models**

I. Introduction

Machine-learning-based network intrusion detection systems commonly represent each network flow as an independent feature vector containing information such as duration, packet counts, byte counts, protocol, and port numbers. Although effective, this formulation does not explicitly model relationships among network hosts.

This limitation is important because many attacks are characterized by communication patterns rather than by a single connection. For example, hundreds of connections from one source to many destinations may indicate scanning or brute-force activity, while many sources simultaneously targeting one destination may indicate a denial-of-service attack. Such behaviors naturally form graph structures.

We therefore formulate intrusion detection as an edge-classification problem. For each time window t, we construct a directed multigraph $G_t = (V_t, E_t)$, where V_t contains observed hosts and each $e \in E_t$ represents a network flow with feature vector $\mathbf{x}_e$. The goal is to predict whether each edge is benign or malicious and, for in-domain experiments, its attack category.

The goal of this project is to develop and evaluate a graph-based intrusion detection framework that combines edge-aware graph learning, cross-dataset generalization, and human-readable alert explanations. Our project investigates four questions: whether graph-based modeling can outperform flow-level classifiers; whether attention-based aggregation improves over mean aggregation; how well the learned model generalizes across datasets; and whether an LLM can convert predictions and graph context into useful human-readable alerts.

II. Related Work

Traditional intrusion detection methods commonly classify individual flows using tabular features, with ensemble methods such as Random Forest and XGBoost remaining strong baselines. However, these approaches largely treat flows independently and therefore do not explicitly model relationships among communicating hosts.

Graph neural networks provide a natural alternative because network traffic can be represented as a communication graph. GraphSAGE [3] learns inductive neighborhood aggregation functions, while graph attention networks [4] allow learned weighting of neighboring information. E-GraphSAGE [1] adapts GraphSAGE for network intrusion detection by aggregating incident edge features and classifying network flows using endpoint embeddings. Anomal-E [2] further demonstrates the usefulness of graph-based edge representations for intrusion detection.

A major limitation of existing graph-based NIDS research is that evaluation is often restricted to a single dataset. Cross-dataset comparison is difficult because intrusion datasets typically use different feature schemas. The standardized NetFlow datasets introduced by Sarhan et al. [5] provide a common representation across multiple datasets and therefore enable direct cross-dataset evaluation. Our project combines this standardized representation with edge-centric graph learning, leakage-aware evaluation, and LLM-generated alert explanations.

III. Dataset and Preprocessing

We use two standardized NetFlow datasets: NF-UNSW-NB15-v2 [6] and NF-CICIDS2018-v3 [7]. The datasets con-

tain approximately 2.39 million and 19.49 million flows, respectively. Both are aligned to 39 common numerical features.

Each graph contains 5,000 consecutive network flows. Unique IP addresses represent nodes, while directed edges represent flows with 39-dimensional features and binary/multiclass attack labels. IP addresses and TTL fields are excluded from predictive features.

CICIDS2018-v3 was deduplicated and chronologically split into training (70%), validation (10%), and testing (20%) sets. UNSW-NB15-v2 was split sequentially using its original record order because flow-start timestamps are unavailable.

Selected numerical features undergo logarithmic transformation and standardization using statistics fitted exclusively on the training split. Evaluation emphasizes macro and per-class F1 scores to address class imbalance.

IV. Proposed Approach

A. Baselines

To evaluate whether graph structure provides additional predictive value beyond flow-level features, we plan to compare the proposed GNN against three non-graph baselines trained using the same input features, data splits, and class-weighting strategy:

1) Random Forest.
2) XGBoost.
3) A multilayer perceptron (MLP) with a parameter count chosen to be reasonably comparable to the GNN.

B. Graph Neural Network Model

Our primary graph model is based on the E-GraphSAGE formulation, which is designed for settings in which informative features are associated primarily with edges rather than nodes. At each message-passing layer, a node updates its representation by aggregating information from the incident network-flow edges:

$$h_v^{(k)} = \sigma\left(W^{(k)} \cdot \mathrm{CONCAT}\left(h_v^{(k-1)}, \mathrm{AGG}_{u \in N(v)}\left(h_{uv}^{(k-1)}\right)\right)\right) \tag{1}$$

where h_{uv} denotes the representation associated with the flow edge between nodes u and v, and AGG denotes a permutation-invariant aggregation function such as mean aggregation.

After K message-passing layers, each flow is classified from the learned embeddings of its source and destination nodes, as in the original E-GraphSAGE model:

$$z_{uv} = \mathrm{CONCAT}\left(h_u^{(K)}, h_v^{(K)}\right) \tag{2}$$

The flow features x_{uv} enter the model through the messages aggregated at each layer. The resulting edge representation is passed to a classification head that outputs a probability distribution over the benign class and the available attack categories. As a variant, the original flow features can also be appended to the edge representation, $z_{uv} = \mathrm{CONCAT}(h_u^{(K)}, h_v^{(K)}, x_{uv})$; comparing the two forms is part of the ablation planned for Checkpoint II.

C. Proposed Extension

Beyond the baseline E-GraphSAGE model, we plan to investigate an attention-based aggregation mechanism. Instead of assigning equal importance to all neighboring flows during message passing, the attention mechanism learns weights that allow the model to emphasize more informative or suspicious connections.

This extension is motivated by the observation that network hosts may participate in many benign connections while only a small subset of their neighboring flows are associated with malicious behavior. Mean aggregation may dilute these important signals, whereas attention-based aggregation may allow the model to focus more strongly on the most relevant communication patterns.

We will compare attention-based aggregation against the original mean aggregation used in the baseline model to evaluate whether learned neighbor weighting improves intrusion detection performance, particularly for minority attack classes.

D. LLM Explanation Layer

As an extension to the core intrusion detection pipeline, we plan to add a large language model (LLM) explanation layer that converts model predictions into short, human-readable security alerts. The LLM will not participate in the detection decision itself; instead, it will receive structured information derived from the GNN output and the corresponding local graph context.

For each flow predicted as malicious, the explanation input will include the predicted attack class, prediction confidence, selected flow features, and local graph statistics such as source-host fan-out, the number of distinct destination ports contacted, and the number of similar flows observed within the same time window. This information will be passed to an LLM using a fixed prompt template through the Virginia Tech ARC LLM Gateway.

The generated explanation will summarize why the flow was flagged in plain language while remaining grounded in the supplied model and traffic evidence. To reduce unsupported explanations, the LLM will be instructed to use only the information provided in the structured input and not introduce additional attack evidence. The explanation component will be evaluated qualitatively for factual consistency and clarity.

E. Evaluation Protocol

- **Primary metrics:** macro F1 and per-class F1.
- **Secondary metrics:** precision, recall, confusion matrices, and precision–recall AUC where appropriate.
- **In-domain evaluation:** held-out train/validation/test splits within each dataset, using chronological splitting where reliable timestamps are available.
- **Cross-dataset evaluation:** train on NF-UNSW-NB15-v2 and test on NF-CICIDS2018-v3, and vice versa, using binary benign-versus-malicious classification to account for differences in attack categories.
- **Ablation studies:** compare graph-based versus flow-level models and attention-based versus mean aggregation.

Additional experiments on flow-window size and class weighting will be considered if time permits.

- **Qualitative evaluation:** manual review of a sample of LLM explanations for factual consistency with the underlying flow.

V. Checkpoint II Progress: Data Preparation and Graph Construction (Shiyi Liu)

A. Completed Work

Implemented preprocessing pipelines for NF-UNSW-NB15-v2 (2.39 million flows) and NF-CICIDS2018-v3 (19.49 million flows). Both datasets were aligned to 39 common numerical features using logarithmic transformations and training-only standardization. CICIDS was deduplicated and chronologically split into training (70%), validation (10%), and testing (20%) sets. UNSW was split sequentially using its original record order.

Constructed directed communication graphs using fixed-size windows of 5,000 flows. Unique IP addresses represent nodes, while edges contain 39-dimensional features and binary/multiclass attack labels. Generated 476 UNSW graphs and 3,896 CICIDS graphs (4,372 total). Verified graph counts and validated the structure and numerical integrity of representative graphs.

B. Checkpoint II Status and Limitations

The data preprocessing and graph construction tasks were completed during Checkpoint I. No major changes were required for Checkpoint II, and the prepared datasets and graph files have been used by other team members for subsequent model training and evaluation.

One limitation remains: UNSW lacks flow-start timestamps, so chronological ordering cannot be verified. Full graph-level validation and leakage auditing have not yet been completed.

VI. Checkpoint II Progress: Flow-Level Baselines and Evaluation (Goku Ravi)

A. Checkpoint I: Baseline Implementation and Pilot

Our objective is to measure what can be learned from individual NetFlow records before introducing graph connectivity. We implemented a shared loader for the preprocessed graph `.npz` files that reads the 39-dimensional `edge_attr` features and `edge_attack` labels, while deliberately excluding `edge_index`. This retains the same train/validation/test partitions used by the graph models. Three non-graph classifiers were implemented: Random Forest (RF), XGBoost, and a PyTorch multilayer perceptron (MLP) with architecture $39 \to 64 \to 32 \to 10$, ReLU activations and dropout. RF uses balanced subsample weights, XGBoost uses per-example weights, and MLP uses class-weighted cross-entropy.

At Checkpoint I, we completed a seed-42 pilot with 20,000 sampled UNSW training flows and the complete 235,000-flow validation split. Table I records these *validation* results; they are not directly comparable to held-out test scores.

TABLE I
Checkpoint I Pilot: UNSW Validation (20K Training Flows)

Metric	RF	XGBoost	MLP
Accuracy	0.9551	0.9550	0.9364
Macro F1	0.3755	**0.4331**	0.2972
Weighted F1	0.9536	**0.9621**	0.9429
Binary attack F1	0.7977	**0.8555**	0.8345

TABLE II
Checkpoint II: UNSW Held-Out Test (475,000 Flows)

Metric	RF	XGBoost	MLP
Accuracy	0.9726	**0.9747**	0.9650
Macro F1	0.6136	**0.6506**	0.5237
Weighted F1	0.9765	**0.9787**	0.9704
Binary F1	0.9533	**0.9548**	0.9262
Binary AP	**0.9783**	0.9498	0.9356

B. Checkpoint II: Full Evaluation Protocol

We extended the pipeline with unified multiclass and binary evaluation, including accuracy, macro/weighted F1, per-class precision and recall, confusion matrices, binary attack-versus-benign F1, and binary average precision (AP). The benign label is determined from each dataset's class mapping rather than assuming a fixed numeric ID. Model configuration, random seed, and predictions are recorded for reproducibility.

On NF-UNSW-NB15-v2, each model used the full 1,670,000-flow training split, with 235,000 validation and 475,000 held-out test flows. On CICIDS2018, training was limited to a stratified sample of 100,000 flows from the 13,640,000-flow training partition; evaluation used the full 1,945,000-flow validation and 3,895,000-flow test partitions. Thus, CICIDS training results must not be described as full-data training. All model results in Tables II and III are held-out *test* measurements.

On UNSW, XGBoost yields the highest macro F1 (0.6506) and binary F1 (0.9548), while RF has the highest binary AP (0.9783). The final E-GraphSAGE model evaluated on the same UNSW test flows obtains macro F1 of 0.511 and binary F1 of 0.953; the flow-level tree ensembles therefore remain competitive or stronger under these settings.

On CICIDS, high accuracy does not imply successful attack detection: approximately 95.64% of test traffic is benign, while RF achieves only 0.0008 binary F1 despite 0.9560 accuracy. The chronological partition contains no *Bot* or *Infilteration* examples in training, although these are the attacks represented in validation and testing. Class weighting cannot recover supervision for entirely missing classes; CICIDS multiclass results are consequently constrained by this split.

C. Imbalance Ablation and Seed Stability

To address highly imbalanced class counts, RF used `balanced_subsample`; XGBoost applied frequency-derived observation weights; and MLP used weighted cross-entropy. We compared weighted and unweighted configurations at a matched 50,000-flow UNSW training

TABLE III
CHECKPOINT II: CICIDS HELD-OUT TEST (3,895,000 FLOWS)

Metric	RF	XGBoost	MLP
Accuracy	**0.9560**	0.9493	0.5891
Macro F1	**0.0652**	0.0650	0.0505
Weighted F1	**0.9349**	0.9331	0.7248
Binary F1	0.0008	0.1223	**0.1549**
Binary AP	0.4960	**0.5157**	0.3439

TABLE IV
UNSW CLASS-WEIGHTING ABLATION: BINARY F1 (50K TRAINING FLOWS)

Model	Without weighting	With weighting
RF	0.641	**0.865**
XGBoost	0.586	**0.904**
MLP	0.411	**0.767**

budget (Table IV). These are separate ablation runs, not the full-data results in Table II.

Weighting improves binary F1 for all three baselines in this controlled comparison. To assess run-to-run variation, RF was also trained with seeds 42, 43, and 44 at the 50K budget, obtaining mean macro F1 of 0.4851 ± 0.0068 (mean $\pm$ standard deviation). Equivalent three-seed studies for XGBoost and MLP remain future work.

D. Zero-Shot Cross-Dataset Generalization

We evaluated source-trained models directly on the opposite dataset's validation partition, without target-domain retraining. To align standardized features, target data were mapped through the target scaler's inverse transform and then the source training scaler. Since attack-category labels differ, transfer is assessed as binary attack-versus-benign classification. Table V reports the resulting binary F1.

All baseline models exhibit substantial performance degradation under domain shift. The poor transfer results should not be attributed to a single cause without further analysis: class availability, traffic composition, feature distributions, and the training budget may all contribute.

E. Summary and Remaining Work

Checkpoint II completed baseline training, held-out evaluation, class weighting ablations, an RF seed study, zero-shot transfer, and a shared reporting pipeline. The central findings are that XGBoost is the best UNSW baseline by macro F1, weighting materially improves minority attack detection in the controlled ablation, and both CICIDS class availability and cross-dataset shift limit generalization. The final report should include detailed per-class confusion matrices and precision–recall plots, additional seeds for XGBoost/MLP, and a revised CICIDS split with all attack types represented in training.

TABLE V
ZERO-SHOT TRANSFER: BINARY F1 (TARGET VALIDATION SPLIT)

Model	UNSW $\rightarrow$ CICIDS	CICIDS $\rightarrow$ UNSW
RF	0.059	0.000
XGBoost	0.053	0.093
MLP	0.053	**0.125**

[x=5.0cm,y=0.52cm] [-¿] (0,0) – (0.75,0) node[right,font=] Macro F1; in 0,0.2,0.4,0.6 (,0.06)–(,-0.06); [below,font=] at (,-0.06) ; [blue!40] (0,0.6) rectangle (0.6136,1.05); [blue!75] (0,1.4) rectangle (0.6506,1.85); [blue!20] (0,2.2) rectangle (0.5237,2.65); [anchor=east,font=] at (-0.015,0.82) RF; [anchor=east,font=] at (-0.015,1.62) XGB; [anchor=east,font=] at (-0.015,2.42) MLP; [anchor=west,font=] at (0.618,0.82) 0.614; [anchor=west,font=] at (0.655,1.62) 0.651; [anchor=west,font=] at (0.528,2.42) 0.524;

Fig. 1. UNSW held-out test macro F1 for the flow-level baselines.

VII. CHECKPOINT II PROGRESS: GRAPH NEURAL NETWORK MODEL (VIGHNESH NAIR)

A. Implementation

E-GraphSAGE [1] is implemented in PyTorch Geometric following Eq. (1): each layer forms a message $m_{uv} = W_{\text{msg}}[h_u \| x_{uv}]$ for every incident flow and updates the host as $h_v = \text{ReLU}(W_{\text{apply}}[h_v \| \text{AGG}_u\, m_{uv}])$. Node inputs are constant, so all information comes from the flows, and each flow also sends a message in the reverse direction, marked by a 0/1 direction flag, so that a host that only sends traffic (for example, a scanner) still aggregates its flows. The classifier uses either the two endpoint embeddings $[h_u \| h_v]$, as in the original model (Eq. (2)), or the variant that appends the flow features x_{uv}. Mean, sum, and attention aggregation are supported; attention weights each incoming message by $\alpha_{uv} = \text{softmax}_u \,\text{LeakyReLU}(a^\top [W_q h_v \| m_{uv}])$, following graph attention networks [4].

The model has two layers with 128 hidden units and dropout 0.2, and is trained with Adam (learning rate 10^{-3}), class-weighted cross-entropy, and early stopping on validation loss. Every run is seeded and defined by a saved configuration file. All reported metrics are computed with the shared evaluation module (Section VI), every selection decision uses the validation split only, and comparisons report the mean $\pm$ standard deviation over three seeds on the held-out test split of NF-UNSW-NB15-v2 (475,000 flows) unless noted. The code and saved predictions are in the team repository.

B. Ablations

Each variant in Table VI changes one component of the published model, keeping inverse-frequency class weights.

The largest effect comes from the classifier input. The published classifier assigns every flow between the same pair of hosts the same prediction, and the UNSW test windows contain on average 34 flows per distinct host pair. Appending x_{uv} raises macro F1 from 0.282 to 0.431; Reconnaissance improves from 0.06 to 0.60 and Exploits from 0.27 to 0.69. The graph still contributes beyond the flow features: with the same classifier and no message passing, macro F1 is 0.359 and

TABLE VI
E-GRAPHSAGE ABLATIONS (NF-UNSW-NB15-V2 TEST, 3 SEEDS)

Variant	Macro F1	Binary F1
Published model ($[h_u \| h_v]$, mean)	0.282 ± 0.008	0.874
Attention aggregation	0.293 ± 0.009	0.874
One-directional messages	0.290 ± 0.018	0.887
One layer	0.287 ± 0.007	0.874
Sum aggregation	0.107 ± 0.009	0.858
No graph (x_{uv} only)	0.359 ± 0.007	0.700
Flow features in classifier	**0.431 ± 0.009**	**0.885**

TABLE VII
CLASS WEIGHTING × CLASSIFIER INPUT (NF-UNSW-NB15-V2, 3 SEEDS)

Input	Weighting	Val Macro F1	Test Macro F1
$[h_u \| h_v]$	None	0.283	0.290 ± 0.001
	Inverse	0.279	0.282 ± 0.008
	Inverse sqrt	0.327	0.337 ± 0.009
$[h_u \| h_v \| x_{uv}]$	None	0.463	0.467 ± 0.014
	Inverse	0.425	0.431 ± 0.009
	Inverse sqrt	**0.506**	**0.511 ± 0.003**

binary F1 falls from 0.885 to 0.700. Attention improves macro F1 by 0.011, within one standard deviation, at the cost of 22% more parameters, so it gives no significant gain on this dataset. Sum aggregation diverged, most likely because unnormalized sums over hundreds of flows per host grow very large; it will be rerun with degree normalization. An architecture sweep over 27 settings (one to three layers, 64 to 256 hidden units, and learning rates of 3×10^{-3}, 10^{-3}, and 3×10^{-4}) changed test macro F1 only from 0.267 to 0.297.

C. Class Weighting and Final Model

Table VII crosses three class-weighting schemes with the two classifier inputs. Full inverse-frequency weights make the rarest class (Worms, 71 training flows) dominate the loss, so it is over-predicted; the inverse square root of the class frequency gives the best validation macro F1 for both inputs. The final model therefore uses $[h_u \| h_v \| x_{uv}]$ with square-root weighting.

Table VIII compares the final model with the flow-level baselines on the same test flows, using their held-out test results from the shared evaluation reports. Tuning nearly doubles the macro F1 of the published model. For benign-versus-attack detection the final GNN is on par with Random Forest (0.953) and close to XGBoost (0.955), but for attack-type classification the tree ensembles remain stronger (macro F1 0.651 for XGBoost versus 0.511). Per class, the final model reaches F1 of 0.996 for Benign, 0.776 for Generic, 0.753 for Exploits, 0.704 for Fuzzers, 0.660 for Reconnaissance, and 0.611 for Shellcode, but only 0.211 for Worms, 0.168 for Backdoor, 0.150 for Analysis, and 0.081 for DoS. Most Analysis flows are predicted as Backdoor (72%), and DoS flows are split between Exploits (38%) and Backdoor (32%). The seed-0 run of the final model (macro F1 0.509, binary F1 0.951) provides the predictions used by the explanation layer.

TABLE VIII
E-GRAPHSAGE AND FLOW-LEVEL BASELINES (NF-UNSW-NB15-V2 TEST)

Model	Macro F1	Accuracy	Binary F1
E-GraphSAGE, published (initial)	0.282	0.931	0.874
E-GraphSAGE, final	**0.511**	**0.969**	**0.953**
Random Forest	0.614	0.973	0.953
XGBoost	0.651	0.975	0.955
MLP	0.524	0.965	0.926

TABLE IX
ZERO-SHOT TRANSFER, BINARY F1 (VALIDATION SPLIT)

Model	UNSW→CICIDS	CICIDS→UNSW
E-GraphSAGE, final	**0.131**	0.049
Random Forest	0.059	0.000
XGBoost	0.053	0.093
MLP	0.053	**0.125**

D. Cross-Dataset Transfer

The model trained on one dataset is applied to the other as benign-versus-attack detection. Following the baseline protocol, target features are moved into the source dataset's feature space by inverting the target scaler and applying the source scaler. Table IX reports binary F1 on the validation split, which matches the split used for the baseline transfer results.

Detection collapses under domain shift for every model: the same GNN reaches a binary F1 of 0.940 on UNSW validation but 0.131 on CIC-IDS2018. From UNSW to CIC-IDS2018 the GNN transfers better than the baselines (0.131 versus at most 0.059; 0.088 on the test split). The reverse direction is limited by the CIC-IDS2018 model itself, described next.

E. Limitation: CIC-IDS2018 Split

With the current chronological split, Bot and Infilteration have no training flows, while the validation and test attacks consist only of these two classes. A GNN trained with the final settings therefore reaches a macro F1 of 0.065 and a binary F1 of 0.001 on the CIC-IDS2018 test set, matching the baselines (macro F1 of about 0.065). This reflects the split rather than the models: meaningful CIC-IDS2018 results require a split in which every attack type appears in training.

F. Status of GNN Tasks

G. Plan for the Final Report

1) Rerun CIC-IDS2018 training and both transfer directions once a split with every attack type in training is available.
2) Evaluate attention aggregation on CIC-IDS2018, whose windows contain about 750 hosts compared with about 30 in UNSW.
3) Rerun sum aggregation with degree normalization.
4) Repeat the transfer experiments over three seeds.
5) Provide final predictions for both datasets to the explanation layer.

TABLE X
GNN TASK STATUS AT CHECKPOINT II

Task	Status
E-GraphSAGE implementation	Done
Training pipeline and shared metrics	Done
Attention vs. mean aggregation (UNSW)	Done
Class-weighting and hyperparameter tuning	Done
Ablations, incl. graph vs. flow-level	Done
Per-class error analysis (UNSW)	Done
Cross-dataset transfer	Done (one seed)
Predictions for the explanation layer	Done
CIC-IDS2018 in-domain results	Final Report

VIII. CHECKPOINT II PROGRESS: EXPLANATION LAYER (KRISHNA INUKONDA)

A. Implementation

The explanation layer converts flows flagged by a detection model into short security alerts. It takes no part in detection. The pipeline reads saved predictions in the shared format together with the corresponding 5,000-flow graphs, and it stops with an error if the predictions and graph labels disagree. For the final UNSW E-GraphSAGE model (macro F1 0.509, binary F1 0.951, one seed), labels matched across all 95 test graphs and 475,000 flows.

Each flagged flow is converted into a structured record with three parts: the model output (predicted class, confidence, and runner-up class); flow fields (hosts, destination port, protocol, bytes and packets in each direction, and TCP flags); and statistics from its window (source outgoing flows, distinct destination hosts and ports, destination incoming flows and distinct sources, repeated flows with the same source, destination, and port, and flagged flow counts). Readable values are recovered from the stored edge features by inverting the training scaler and log transform. Ground-truth labels and IP addresses are never sent to the LLM; hosts appear only as per-window indices.

Records are sent to `gpt-oss-120b` through the Virginia Tech ARC LLM Gateway using a fixed prompt and a temperature of 0.2, and responses are cached so that reported alerts can be reproduced. The prompt requires numbers to be copied exactly, forbids speculation about intent, payloads, or remediation, and requires low-confidence predictions (below 60%) to be labeled uncertain. Alerts describe evidence as consistent with a prediction rather than as the reason for it, because the model's internal reasoning is not available to the LLM.

B. Evaluation Protocol

The sample contains 54 alerts: two flows for each combination of nine predicted attack classes and three outcomes (correct class, wrong attack class, and false positive). Mistakes are included deliberately. Of the 38,325 flows the final model flagged on the UNSW test set, 61% received the correct attack class, 9% were benign, and 40% had confidence below 60%, so an alert must remain accurate when the underlying prediction is wrong. Automated checks flag numbers absent from the input, unsupported attack classes, speculative terms, and format violations. Each alert was then reviewed against its structured input for factual consistency and clarity.

C. Data Issue and Prompt Refinement

Review of the first run exposed a data-quality problem. In the UNSW test split, `FLOW_DURATION_MILLISECONDS` is exactly zero for 93.7% of flows, and every remaining value is at least 4,290,000 ms, consistent with a 32-bit NetFlow timer wraparound. Duration is therefore withheld from the LLM.

TABLE XI
ALERTS CONTAINING EACH ERROR ACROSS PROMPT VERSIONS ($n = 54$ PER VERSION)

Error	v1	v2	v3
Invalid duration quoted	9	0	0
Implied time order	21	0	1
False "only flagged flow" claim	2	0	0
Incorrect host-and-port count	0	14	0
Mean alert length (words)	78	94	73

Table XI summarizes three prompt versions. The 14 host-and-port errors in v2 came from an ambiguous field name in the structured input, which described flows to a port on any host; the LLM reported them as flows to one host. The field was replaced with a count matching that wording. Automated checks passed 53 of 54 alerts in v1 and 54 of 54 in v3, although both versions contained factual errors. The checks can confirm that a number appears in the input, but not that it is attached to the right fact, so they complement rather than replace review of the alerts.

D. Results

TABLE XII
REVIEW OF FINAL ALERTS (PROMPT V3)

Outcome	Alerts	Factually consistent	Clear
Correct class	18	18	16
Wrong attack class	18	17	17
False positive	18	16	17
Total	**54**	**51**	**50**

Of the final alerts, 51 of 54 were factually consistent with their input and 50 of 54 were clear (Table XII). All 31 alerts below 60% confidence stated that the category was uncertain and named the runner-up class. The three factual errors were one incorrect statement that byte counts were equal in both directions, one implied ordering ("the second flow"), and one extension of a flow's protocol to other flows.

A consistent alert does not imply a correct detection, as the following false positive shows: *"Flagged as Shellcode (99% confidence). The flow from host_11 to host_21 used TCP port 55117, transferred 88 bytes in 2 packets from source to destination, and only the SYN flag was seen."* The alert accurately reports the evidence, but the flow is benign.

Confidence therefore needs to be interpreted with care, and alerts should support analyst review rather than replace it.

E. Limitations

These results come from 54 alerts, one LLM, one detection model, one seed, and the UNSW test set. Factual consistency measures agreement between an alert and its input; it does not establish that the alert reflects the features the GNN actually used.

F. Status of Explanation and Visualization Tasks

TABLE XIII
EXPLANATION, VISUALIZATION, AND DEMO TASK STATUS AT CHECKPOINT II

Task	Status
Project website deployment and maintenance	Ongoing
Downstream data contract & interface design	Done
LLM prompt template & ARC Gateway prototype	Done
Subgraph context extraction from GNN outputs	Done
Qualitative evaluation of LLM explanations	Done (54 alerts)
Visualization dashboard & final demo	Final Report

G. Plan for the Final Report

1) Build the visualization dashboard showing selected communication graphs, flagged flows, confidence scores, and their alerts.
2) Expand the review to a larger sample, and compare alert quality across LLMs if time permits.
3) Generate alerts for NF-CICIDS2018-v3 once final GNN predictions are available for that dataset.
4) Record the demonstration video.

IX. SYSTEM DESIGN

The system consists of four stages: preprocessing and graph construction, model training, inference and explanation, and visualization. Models will be implemented using PyTorch and PyTorch Geometric and trained using Virginia Tech computing resources.

The overall pipeline is:

$$\text{NetFlow Data} \rightarrow \text{Graph Construction} \rightarrow \text{GNN/Baselines} \rightarrow \text{Attack Prediction} \rightarrow \text{LLM Explanation}$$

If time permits, a lightweight dashboard will visualize selected communication graphs, malicious-flow predictions, confidence scores, evaluation results, and LLM-generated explanations.

REFERENCES

[1] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2022.

[2] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, "Anomal-E: A self-supervised network intrusion detection system based on graph neural networks," *Knowledge-Based Systems*, 2022.

[3] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[5] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," in *Big Data Technologies and Applications*, Springer, 2021.

[6] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. Military Communications and Information Systems Conference (MilCIS)*, 2015.

[7] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.

TABLE XIV
TASK ASSIGNMENT

Member	Responsibilities
Shiyi Liu	Dataset preparation, preprocessing, temporal splitting, and graph construction.
Gokulakrishnan Ravi	Random Forest, XGBoost, MLP baselines, and the shared evaluation metrics (macro and per-class F1).
Vighnesh Nair	E-GraphSAGE, attention-based extension, hyperparameter tuning and class weighting, ablations, and cross-dataset experiments.
Krishna Inukonda	LLM explanation module, dashboard, project website, and demonstration video.
All members	Proposal/report writing and checkpoint/final presentations.

TABLE XV
PROJECT SCHEDULE

Period	Milestone
Sep 15–Oct 6	Dataset preprocessing, graph construction, and baseline models.
Oct 7–20	Initial E-GraphSAGE implementation and evaluation.
Oct 21–Nov 10	Attention extension, cross-dataset experiments, and LLM prototype.
Nov 11–Dec 11	Ablation studies, dashboard, final experiments, presentation, and final report.